

Exponent CLMM

Smart Contract Security Assessment

November 2025

Prepared for:

Exponent Finance

Prepared by:

Offside Labs

Zhoujie Tong

Tim Li





Contents

1	About Offside Labs	2
2	Executive Summary	3
3	Summary of Findings	5
4	Key Findings and Recommendations	7
4.1	Inaccurate CLMM Calculation Formula Leading to Inequitable Outcomes and Swap Inconsistencies	7
4.2	Missing Token Transfer in trade_pt for SyToPt Case	8
4.3	Incorrect PT Transfer Amount During Trade PT	9
4.4	Admin validation inverted in MarketThree::validate_admin	10
4.5	Incorrect Use of Logarithmic Implied Rate as Price in CLMM Calculations	11
4.6	Incorrect Delta Price Calculation in Swap Logic	13
4.7	Fees are not Correctly Transferred to Users During Liquidity Withdrawal	14
4.8	Fee-inside computation compares RB-tree indices instead of tick keys	14
4.9	Improper Tick-Cross Trigger in Swap	15
4.10	Clear-Then-Check on ticks Enables Cross-Market State Overwriting	17
4.11	Incorrect Parameter Usage Leads to Inaccurate Liquidity Calculation	18
4.12	Incorrect Tick Range Traversal During Liquidity Addition	18
4.13	Incorrect Initialization of current_tick Leads to Malformed Liquidity Boundaries	19
4.14	Incorrect fee_growth_outside Initialization	20
4.15	Incorrect Subtree Sum Update During Tree Rotations	21
4.16	Inconsistent Emissions Tracking Mechanism in deposit_liquidity IX	22
4.17	New Ticks Start With Zero Principals and Do Not Split Existing Interval Principals	23
4.18	Fixed-size Tick Storage Enables DoS via Junk Tick Initialization	25
4.19	Incorrect Strip Amount Calculation in WrapperProvideLiquidity IX	25
4.20	Incorrect fee_growth_index_global Update when Ticks Add Fee Growth	26
4.21	Missing mint_yt Validation During Market Initialization	27
4.22	Unused PT and SY Token Transfers During Initialization	28
4.23	Informational and Undetermined Issues	29
5	Disclaimer	32



1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers*, *operating systems*, *IoT devices*, and *hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple*, *Google*, and *Microsoft*, have protected digital assets valued at over **\$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **\$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.



<https://offside.io/>



<https://github.com/offsidelabs>



https://twitter.com/offside_labs



2 Executive Summary

Introduction

Offside Labs completed a security audit of *Exponent-CLMM* smart contracts, starting on September 22th, 2025, and concluding on October 10th, 2025.

Project Overview

The `exponent_clmm` program implements a Concentrated Liquidity Market Maker (CLMM), a capital-efficient decentralized exchange. It is a core component of the Exponent protocol, designed to create a liquid market for the trading of Principal Tokens and SY Tokens that are generated by the `exponent_core` program.

This program is a sophisticated and feature-rich CLMM that is tightly integrated with the broader Exponent protocol. Its design emphasizes capital efficiency, composability, and user experience through wrapper functions.

Audit Scope

The assessment scope contains mainly the smart contracts of the CLMM program for the *Exponent-CLMM* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- Exponent-CLMM
 - Codebase: <https://github.com/exponent-finance/exponent-core>
 - Commit Hash: 4cdd8f174c39d7391169a887b4446ff235615a37

We listed the files we have audited below:

- Exponent-CLMM
 - `solana/programs/exponent_clmm/src/instructions/core_cpi.rs`
 - `solana/programs/exponent_clmm/src/instructions/market_three/admin/add_farm.rs`
 - `solana/programs/exponent_clmm/src/instructions/market_three/admin/add_market_emission.rs`
 - `solana/programs/exponent_clmm/src/instructions/market_three/admin/market_three_init.rs`
 - `solana/programs/exponent_clmm/src/instructions/market_three/admin/modify_farm.rs`
 - `solana/programs/exponent_clmm/src/instructions/market_three/admin/modify_market_setting.rs`
 - `solana/programs/exponent_clmm/src/instructions/market_three/buy_yt.rs`
 - `solana/programs/exponent_clmm/src/instructions/market_three/claim_farm_emissions.rs`



- solana/programs/exponent_clmm/src/instructions/market_three/deposit_liquidity.rs
- solana/programs/exponent_clmm/src/instructions/market_three/market_collect_emission.rs
- solana/programs/exponent_clmm/src/instructions/market_three/sell_yt.rs
- solana/programs/exponent_clmm/src/instructions/market_three/trade_pt.rs
- solana/programs/exponent_clmm/src/instructions/market_three/withdraw_liquidity.rs
- solana/programs/exponent_clmm/src/instructions/self_cpi.rs
- solana/programs/exponent_clmm/src/instructions/util.rs
- solana/programs/exponent_clmm/src/instructions/wrappers/buy_pt.rs
- solana/programs/exponent_clmm/src/instructions/wrappers/sell_pt.rs
- solana/programs/exponent_clmm/src/instructions/wrappers/wrapper_buy_yt.rs
- solana/programs/exponent_clmm/src/instructions/wrappers/wrapper_provide_liquidity.rs
- solana/programs/exponent_clmm/src/instructions/wrappers/wrapper_provide_liquidity_classic.rs
- solana/programs/exponent_clmm/src/instructions/wrappers/wrapper_sell_yt.rs
- solana/programs/exponent_clmm/src/instructions/wrappers/wrapper_withdraw_liquidity.rs
- solana/programs/exponent_clmm/src/instructions/wrappers/wrapper_withdraw_liquidity_classic.rs
- solana/programs/exponent_clmm/src/lib.rs
- solana/programs/exponent_clmm/src/state/cpi_common.rs
- solana/programs/exponent_clmm/src/state/market_three/helpers/add_liquidity.rs
- solana/programs/exponent_clmm/src/state/market_three/helpers/remove_liquidity.rs
- solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs
- solana/programs/exponent_clmm/src/state/market_three/market_three.rs
- solana/programs/exponent_clmm/src/state/position.rs
- solana/programs/exponent_clmm/src/state/ticks.rs
- solana/programs/exponent_clmm/src/utils/math.rs

Findings

The security audit revealed:

- 4 critical issues
- 6 high issues
- 7 medium issues
- 5 low issues
- 8 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.



3 Summary of Findings

ID	Title	Severity	Status
01	Inaccurate CLMM Calculation Formula Leading to Inequitable Outcomes and Swap Inconsistencies	Critical	Fixed
02	Missing Token Transfer in trade_pt for SyToPt Case	Critical	Fixed
03	Incorrect PT Transfer Amount During Trade PT	Critical	Fixed
04	Admin validation inverted in MarketThree::validate_admin	Critical	Fixed
05	Incorrect Use of Logarithmic Implied Rate as Price in CLMM Calculations	High	Fixed
06	Incorrect Delta Price Calculation in Swap Logic	High	Fixed
07	Fees are not Correctly Transferred to Users During Liquidity Withdrawal	High	Fixed
08	Fee-inside computation compares RB-tree indices instead of tick keys	High	Fixed
09	Improper Tick-Cross Trigger in Swap	High	Fixed
10	Clear-Then-Check on ticks Enables Cross-Market State Overwriting	High	Fixed
11	Incorrect Parameter Usage Leads to Inaccurate Liquidity Calculation	Medium	Fixed
12	Incorrect Tick Range Traversal During Liquidity Addition	Medium	Fixed
13	Incorrect Initialization of current_tick Leads to Malformed Liquidity Boundaries	Medium	Fixed
14	Incorrect fee_growth_outside Initialization	Medium	Fixed
15	Incorrect Subtree Sum Update During Tree Rotations	Medium	Fixed
16	Inconsistent Emissions Tracking Mechanism in deposit_liquidity IX	Medium	Fixed
17	New Ticks Start With Zero Principals and Do Not Split Existing Interval Principals	Medium	Fixed
18	Fixed-size Tick Storage Enables DoS via Junk Tick Initialization	Low	Acknowledged
19	Incorrect Strip Amount Calculation in WrapperProvideLiquidity IX	Low	Fixed
20	Incorrect fee_growth_index_global Update when Ticks Add Fee Growth	Low	Fixed



ID	Title	Severity	Status
21	Missing mint_yt Validation During Market Initialization	Low	Fixed
22	Unused PT and SY Token Transfers During Initialization	Low	Fixed
23	RemoveMarketEmission Lacks Validation	Informational	Acknowledged
24	Emission Processing Lacks Specific Support for Token2022 Transfer Fees and Memo	Informational	Acknowledged
25	Unhandled is_current_flash_swap Flag in Swap Logic	Informational	Acknowledged
26	Incorrect Calculation of Additional Rent for Account Resizing	Informational	Acknowledged
27	Division by Zero in Percentage Increase Calculation	Informational	Acknowledged
28	lp_position Initialization Keyword Restricts Multiple Liquidity Deposits	Informational	Acknowledged
29	User lp_position Accounts Are Never Closed	Informational	Acknowledged
30	Incorrect SIZEOF for ConfigurationOptions	Informational	Acknowledged



4 Key Findings and Recommendations

4.1 Inaccurate CLMM Calculation Formula Leading to Inequitable Outcomes and Swap Inconsistencies

Severity: Critical

Status: Fixed

Target: Smart Contract

Category: Math Error

Description

The current CLMM implementation utilizes `effect_price`, `liquidity`, and `sy_amount` in its core calculation formulas for both `add_liquidity` and `swap` operations. A critical issue arises because `effect_price` is influenced by `time_factor` and `sy_exchange_rate`, which in turn directly impacts the calculated liquidity. This dynamic introduction of time- and exchange-rate-dependent variables into the core liquidity calculation creates two significant problems:

1. Inconsistent Liquidity Provision: Users depositing the same `sy_amount` at different times will receive varying amounts of liquidity due to fluctuations in `effect_price`. However, during withdrawal, the system calculates the user's share of `sy_amount` based on their proportion of the tick's total liquidity. This discrepancy means that users who deposited at times when `effect_price` yielded higher liquidity for the same `sy_amount` will disproportionately receive a larger share of SY tokens upon withdrawal, creating an unfair distribution among liquidity providers.
2. Swap Inconsistencies and Price Gaps: The adjustment of prices due to `effect_price` in swap calculations can lead to situations where a tick's token balance is insufficient to fulfill a trade at the calculated price, or conversely, has an unexpected surplus. When a tick's balance is insufficient, the swap operation might "skip" a portion of the price range and "cross tick" to the next available tick. However, when an opposite trade is initiated, the price calculation commences from the *boundary* of the tick, not necessarily accounting for the previously skipped range or the adjusted `effect_price`. This asymmetry can lead to substantial differences in the amount of tokens received or provided for two transactions that are conceptually "opposite" in direction, creating significant slippage and an inefficient trading experience.

Impact

The primary impact is a severe erosion of fairness for liquidity providers. Those who deposit at less opportune times receive a smaller share of rewards relative to their contribution, which can lead to a loss of trust and reduced participation. For traders, the inconsistent swap behavior introduces unpredictable slippage and potentially unfavorable execution prices, hindering efficient market making and trading strategies. Overall, the protocol's capital efficiency and reliability are compromised by these fundamental calculation inaccuracies.



Recommendation

It is imperative to meticulously review and revise the CLMM calculation formulas to ensure consistency and integrity, particularly concerning the role of `effect_price`. The calculation of liquidity based on `sy_amount` should ideally be independent of fluctuating parameters like `time_factor` and `sy_exchange_rate` to ensure that equal `sy_amount` contributions always yield equal liquidity shares. If `effect_price` must influence liquidity, a robust mechanism for proportional allocation and withdrawal that accurately reflects each user's true contribution, regardless of when it occurred, needs to be implemented. Furthermore, the swap mechanism needs to be re-evaluated to prevent "tick skipping" and ensure that forward and reverse trades within the same price range result in consistent and predictable outcomes, possibly by standardizing how prices are consumed and reset across tick boundaries. This revision should prioritize mathematical soundness and equitable treatment for all participants.

Mitigation Review Log

Fixed in commit bc24230167e32fa6a4ac62656ae05be9ad0d8762.

4.2 Missing Token Transfer in `trade_pt` for SyToPt Case

Severity: Critical

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

Within the `trade_pt` IX, specifically when handling the `SyToPt` trade scenario, the `transfer_sy` function is called. However, during this call, only a `deposit_sy` operation is performed, which transfers tokens from `escrow_sy`. Crucially, there is no corresponding `token_transfer` executed to deduct the SY tokens from the user's actual token account. This means that while the protocol's internal `escrow_sy` balance is updated, the user's external token balance remains unchanged, effectively allowing them to acquire PT tokens without relinquishing the required SY tokens.

Impact

This oversight is a critical vulnerability that can lead to:

- Asset Drain: Users can effectively mint PT tokens by initiating trades from SY without actually paying for them in SY, leading to a loss of value for the protocol and other users.
- Protocol Solvency Issues: The protocol's `escrow_sy` balance will be reduced as if a valid trade occurred, but the corresponding SY tokens were never debited from the user, leading to an imbalance and potential insolvency.



- Inaccurate Balances: The on-chain state of user SY token balances will not reflect their actual expenditures within the `trade_pt` function.
- Exploitation Risk: This is a severe bug that could be easily exploited by malicious actors to drain the protocol's PT reserves.

Recommendation

Ensure that when `transfer_sy` is called in the `trade_pt` IX for the `SyToPt` case, a proper `token_transfer` mechanism is invoked to move the required Sy tokens from the user's account to the `escrow_sy`.

The corrected logic should include both the deposit management and the token movement from the user's wallet.

Mitigation Review Log

Fixed in commit 92a3cb95bda999fb876232fcb0422274d8a6893b.

4.3 Incorrect PT Transfer Amount During Trade PT

Severity: Critical

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

In the `trade_pt` IX, `do_transfer_pt` is responsible for handling the transfer of PT tokens, either from the user to an escrow account or vice-versa, depending on the trade direction. However, the current implementation passes `trade_result.amount_out` as the transfer amount parameter to `do_transfer_pt`. This design flaw means that `do_transfer_pt` will consistently attempt to transfer `amount_out`, even when the trade is `PtToSy` (meaning the user is sending PT tokens and receiving SY tokens). In a `PtToSy` trade, the user should be sending `amount_in` PT tokens, not `amount_out`.

```
243     ctx.accounts
244         .do_transfer_pt(trade_result.amount_out, swap_direction,
    → trade_result.protocol_fee_charged_out_token)?;
```

[solana/programs/exponent_clmm/src/instructions/market_three/trade_pt.rs#L243-L244](https://github.com/exponent-clmm/solana/programs/exponent_clmm/src/instructions/market_three/trade_pt.rs#L243-L244)

Impact

This incorrect parameter usage leads to a discrepancy where the amount of PT tokens transferred into the project's escrow account is less than the actual `amount_in` that the user committed for a `PtToSy` trade. Consequently, the project's escrow will incur losses, as it will



receive fewer PT tokens than it should, failing to cover the SY tokens minted for the user. Over time, this can lead to a significant deficit in the project's reserves and financial instability.

Recommendation

It is crucial to correct the parameter passed to `do_transfer_pt` within the `trade_pt` IX. The transfer amount should be dynamically determined based on the trade direction. Specifically, for a `PtToSy` trade, `do_transfer_pt` should be called with `trade_result.amount_in` as the transfer amount for PT tokens. For `SyToPt` trades, where PT is being sent to the user, `trade_result.amount_out` would be appropriate. This ensures that the correct quantity of PT tokens is transferred, safeguarding the project's escrow from unintended losses.

Mitigation Review Log

Fixed in commit 1905065982ab7f782ca946d5f672b7aead8d51a1.

4.4 Admin validation inverted in `MarketThree::validate_admin`

Severity: Critical

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

`MarketThree::validate_admin` rejects the admin and allows non-admins. Current code:

```
173     pub fn validate_admin(&self, signer: Pubkey) -> Result<()> {
174         if self.admin == signer {
175             return err!(ExponentCLMMError::Unauthorized);
176         }
177
178         Ok(())
179     }
```

[solana/programs/exponent_clmm/src/state/market_three/market_three.rs](#)
#L173-L179

This is inverted: when `signer` equals `self.admin`, it errors; otherwise it passes.

Impact

- Any instruction that relies on `validate_admin` (e.g., admin-only config/farm updates) can be executed by any user, enabling unauthorized parameter changes or fund flows.
- The real admin is blocked from performing management actions.



Recommendation

It is recommended to return an `Unauthorized` error when `self.admin != signer` to ensure proper access control validation.

Mitigation Review Log

Fixed in commit `4e7370db4a937a93e78d22142d0af547ecd5089c`.

4.5 Incorrect Use of Logarithmic Implied Rate as Price in CLMM Calculations

Severity: High

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

During CLMM calculations, specifically for `add_liquidity` (using `lower_price`, `upper_price`) and `swap` operations (using `anchor_u_left`, `anchor_u_right`), the system retrieves a value using `get_ln_implied_rate`. This function returns `self.ticks_tree.get_node(idx).value.ln_implied_rate`. Upon inspection of the Tick structure, `ln_implied_rate` is defined as $(1.0 + (\text{key as f64}) / \text{TICK_KEY_BASE_POINTS}).\ln()$, which is explicitly the natural logarithm of an implied rate.

```
382     pub fn get_ln_implied_rate(&self, idx: u32) -> f64 {
383         self.ticks_tree.get_node(idx).value.ln_implied_rate
384     }
```

[solana/programs/exponent_clmm/src/state/ticks.rs#L382-L384](#)

```
161     let tick = Tick {
162         liquidity_net: 0,
163         liquidity_gross: 0,
164         fee_growth_outside_pt: self.fee_growth_index_global_pt, //
    ← snapshot at creation
165         ln_implied_rate: (1.0 + (key as f64) / TICK_KEY_BASE_POINTS).ln(),
166         subtree_sum: 0, // will be recomputed upwards
167         principal_pt: 0,
168         principal_sy: 0,
169         fee_growth_outside_sy: self.fee_growth_index_global_sy,
170     };
```

[solana/programs/exponent_clmm/src/state/ticks.rs#L161-L170](#)

The critical flaw lies in the subsequent use of this `ln_implied_rate` value. Instead of converting it back to a linear price, the system directly uses this logarithmic value as if



it were a linear price in CLMM formulas. This includes calculating `delta_price` and applying it within the core CLMM mechanics. This approach fundamentally misinterprets the nature of `ln_implied_rate`, which is a log-transformed value, not the linear price itself.

Impact

Directly using a logarithmic value where a linear price is expected in CLMM calculations will lead to significant inaccuracies across all affected operations.

1. **Incorrect Liquidity Calculations:** Both adding and removing liquidity will be based on distorted price ranges, causing miscalculations of `liquidity_net` and `liquidity_gross`. This will result in users providing or withdrawing incorrect amounts of tokens relative to the actual price and their intended liquidity range.
2. **Erroneous Swap Execution:** Swaps will operate with an incorrect understanding of the current price and price boundaries. This will lead to improper calculation of `amount_in` and `amount_out`, excessive slippage, and potential arbitrage opportunities arising from the mispriced trades.
3. **Violated Invariants:** The fundamental invariants of the CLMM model, which rely on linear price relationships for correct token accounting and liquidity management, will be violated. This can lead to imbalances in pool reserves and instability.
4. **Misrepresentation of Price:** The entire system will operate on a “logarithmic price” internally for critical operations, which is not how market prices are understood or typically used in financial models, leading to a system that behaves unpredictably and incorrectly.

Recommendation

It is imperative to correct how `ln_implied_rate` is utilized in all CLMM calculations. Before using `ln_implied_rate` in any formula that expects a linear price (e.g., `lower_price`, `upper_price`, `anchor_u_left`, `anchor_u_right`), it must be converted back to its linear form by applying the exponential function (e^x). Specifically, any part of the calculation that uses `get_ln_implied_rate` as a price should perform `exp(get_ln_implied_rate(idx))` to obtain the correct linear price. This fundamental correction is essential to ensure the CLMM operates according to its designed mathematical principles and accurately reflects market dynamics.

Mitigation Review Log

Fixed in commit 537aa5842851cfdb87d0f7cbf47abe6321266d8a.



4.6 Incorrect Delta Price Calculation in Swap Logic

Severity: High

Status: Fixed

Target: Smart Contract

Category: Math Error

Description

The current swap logic in the protocol exhibits an incorrect calculation for boundary price differences (`du_to_boundary` and `delta_c_to_left_boundary`), failing to align with the underlying liquidity addition model. According to the liquidity provision logic, adding SY occurs below the `current_tick` , while adding PT occurs above it. This means that a `PtToSy` trade should inherently lead to a price decrease. Conversely, a `SyToPt` trade should cause a price increase.

However, the existing implementation for `PtToSy` trades calculates `du_to_boundary` as `anchor_u_right - ticks.current_spot_price` . This is inverted; given the expected price decrease, the calculation should be `current_price - anchor_u_left` . Similarly, for `SyToPt` trades, where an increase in price is expected, the calculation for `delta_c_to_left_boundary` is also misaligned and should instead use `c_eff_old - c_eff_right` . These discrepancies indicate a misunderstanding of how price movement interacts with tick boundaries in relation to the specific trade directions.

Impact

This fundamental miscalculation directly results in incorrect pricing for swaps. The protocol will fail to accurately determine the effective price at which a trade should execute, leading to trades settling at values that do not reflect the true market conditions or the available liquidity within tick ranges.

Recommendation

The swap logic must be revised to correctly calculate `du_to_boundary` and `delta_c_to_left_boundary` based on the expected price movement for each trade direction. For `PtToSy` trades (price decrease), `du_to_boundary` should be `current_price - anchor_u_left` . For `SyToPt` trades (price increase), `delta_c_to_left_boundary` should be `c_eff_old - c_eff_right` . These adjustments are necessary to ensure accurate swap calculations aligned with the underlying liquidity model.

Mitigation Review Log

Fixed in commit `fb0f4babb919b54d14a802148068bffa8bf75fce`.



4.7 Fees are not Correctly Transferred to Users During Liquidity Withdrawal

Severity: High

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

During the `withdraw_liquidity` process, the current implementation incorrectly zeroes out the fee value after the liquidity withdrawal without transferring the accumulated fees to the user. This means that while fees are being calculated, they are not actually being disbursed, leading to a loss of expected revenue for users and an accumulation of untransferred funds within the system.

Impact

This oversight results in users not receiving the fees they are entitled to from their liquidity contributions. This can significantly reduce user trust, potentially deterring participation in the liquidity pool. Furthermore, untransferred fees may accumulate, creating a pool of funds that are unaccounted for in the system's operational logic, which could lead to discrepancies in financial reporting and potential legal liabilities.

Recommendation

It is recommended to implement a mechanism within the `withdraw_liquidity` function to ensure that the accumulated fee amount is correctly calculated and transferred to the user's address before the fee value is reset or cleared. This should involve verifying the user's share of the fees and executing a transfer transaction to disburse these funds appropriately.

Mitigation Review Log

Fixed in commit 71460323dd7ff23ffa3b931c49c5f1e282f62081.

4.8 Fee-inside computation compares RB-tree indices instead of tick keys

Severity: High

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

`compute_fee_inside_for_range` determines the fee "inside" a position by comparing:



```
if current < lower_idx {  
    ...  
} else if current >= upper_idx {  
    ...  
} else {  
    ...  
}
```

Here `current` , `lower_idx` , and `upper_idx` are node indices (RB-tree positions), not tick keys/prices. RB-tree indices have no monotonic relation to price order; only the nodes' `key` (or derived price) reflects ordering. Using indices can select the wrong branch and produce incorrect inside fee growth.

Impact

- Mis-accounted fee growth “inside” ranges -> under/over-payment to LPs.
- Potential fee theft or denial of fees when tree rotations/insertions change node indices.

Recommendation

Compared by tick key, not not by node index.

Mitigation Review Log

Fixed in commit 4dd5b4f741361a56ef9d4d6c48094bb4dcc6077f.

4.9 Improper Tick-Cross Trigger in Swap

Severity: High

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

Inside the swap loop, the code decides to cross a tick when the computed movement within the current interval is “too small”:



```
122     if du_actual <= self.configuration_options.epsilon_clamp {
123         // Cross boundary to unlock the next interval
124         self.cross_one_boundary_and_update_core(
125             ticks,
126             SwapDirection::PtToSy,
127             right_boundary_index,
128             &mut current_left_boundary_index,
129             &mut current_price_spot,
130             &mut active_liquidity_u64,
131             &mut active_liquidity_f64,
132             mutate_tick_state,
133         )?;
134         continue;
135     }
```

[solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs](#)
#L122-L135

```
244     if delta_c_actual <= eps {
245         // Cross boundary to the left
246         self.cross_one_boundary_and_update_core(
247             ticks,
248             SwapDirection::SyToPt,
249             current_left_boundary_index, // crossing the current lower
250             &mut current_left_boundary_index,
251             &mut current_price_spot,
252             &mut active_liquidity_u64,
253             &mut active_liquidity_f64,
254             mutate_tick_state,
255         )?;
256         continue;
257     }
```

[solana/programs/exponent_clmm/src/state/market_three/helpers/swap.rs](#)
#L244-L257

This is unreasonable. `epsilon_clamp` is a *numerical precision* guard for “make no move” cases, not a *routing rule* for when to cross price intervals. A tiny input (or tight principal clamp) can make `du_actual / delta_c_actual ~ 0` even when the current price is not near the boundary. The code then crosses anyway, which:

- Skips execution in the present interval,
- Jumps into the next interval without having consumed meaningful input or reached the boundary,
- Causes unintended price jumps and inconsistent accounting.



Impact

Attackers can use this to manipulate the price by letting `amount_in_left` be small.

Recommendation

Cross only when at the boundary, replace the condition `if du_actual <= epsilon_clamp { cross... }` with `if du_to_boundary <= boundary_epsilon { cross... }` and analogously for the SY PT branch.

Mitigation Review Log

Fixed in commit 63da6a70d91fd59d1c093a67a5f9448eb278c7ad.

4.10 Clear-Then-Check on ticks Enables Cross-Market State Overwriting

Severity: High

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

`ticks` is passed as an `UncheckedAccount` and its data is zeroed and discriminator written before validation. Subsequent checks in `Ticks::init` (e.g., `market == Pubkey::default()` and allocator “not initialized”) then trivially pass because the state was just cleared. This allows rewriting a previously initialized `ticks` (e.g., another market’s ticks).

Impact

Cross-Market State Overwriting: Because `ticks` is zeroed before validation, an attacker can supply any program-owned, writable, same-size account (e.g., another market’s `ticks`) and have it re-initialized to point at a different market. Since the `init` instruction is currently permissionless, anyone can trigger this across markets, making the impact severe.

Recommendation

The `init` instruction should be admin-only (as in `MarketTwo`). If permissionless creation is intentional, then at minimum validate `ticks` first before writing any bytes.

Mitigation Review Log

Fixed in commit faaa790ce5f5e3342d214637c957b6f80848aede.



4.11 Incorrect Parameter Usage Leads to Inaccurate Liquidity Calculation

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

The `active_liquidity_u64` function within the tick module is designed to calculate liquidity for the current tick. However, it incorrectly passes `current_tick` as the `target_key` argument to the `prefix_sum` function. The `current_tick` variable represents the red-black tree's `node_idx`, whereas the `prefix_sum` function expects the actual tick value.

Impact

This misuse of parameters leads to an erroneous calculation of liquidity during swap operations. Consequently, the protocol will operate with an inaccurate understanding of available liquidity at a given tick, resulting in price deviations and potentially unfavorable exchange rates for users. This could lead to financial losses for traders and destabilize the automated market maker's pricing mechanism.

Recommendation

The `active_liquidity_u64` function should be modified to correctly derive and pass the actual tick value to the `prefix_sum` function, instead of the `node_idx`. Ensure that `prefix_sum` receives the precise `target_key` it requires to accurately sum liquidity up to that specific tick.

Mitigation Review Log

Fixed in commit `de9ffe0c1579d144dbd9c914e5b16aaa68fc6eb8`.

4.12 Incorrect Tick Range Traversal During Liquidity Addition

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

In the `add_liquidity` process, the function `accrue_principal_for_deposit` is responsible for updating the liquidity-related values within the relevant ticks. However, the implementation traverses an incorrect range of ticks during this update.



As a result, the principal values are added to the wrong ticks or are not updated where they should be. This misalignment occurs because the iteration boundaries or conditions used for tick traversal do not match the intended tick range associated with the liquidity position being added.

Impact

This logic error causes inaccurate liquidity accounting within the affected tick ranges. Consequently:

- Liquidity amounts recorded in the pool become inconsistent with the actual state.
- Subsequent operations, such as swap, may fail or produce incorrect results due to corrupted tick data.
- In severe cases, this could cause the pool to become unusable or result in unexpected fund misallocations between liquidity providers.

Recommendation

Review and correct the tick traversal logic in `accrue_principal_for_deposit` to ensure it only iterates through the intended tick range for the current liquidity position.

Specifically:

1. Verify that the traversal boundaries align with the position's lower and upper ticks.
2. Add explicit checks or assertions to ensure tick indexes fall within valid ranges.
3. Implement unit tests that simulate liquidity addition at various tick ranges and confirm accurate updates to tick data.

Mitigation Review Log

Fixed in commit `e5a22a17a84a9539c4ae2638ad0b7f6d31fe3207`.

4.13 Incorrect Initialization of `current_tick` Leads to Malformed Liquidity Boundaries

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

The `current_tick` variable is being initialized at an inappropriate stage within the program's lifecycle. Specifically, `current_tick` is set during the `upsert_boundary_tick` function, which is subsequently called within `add_liquidity`. The correct approach dictates that `current_tick` should be established during the initial setup of `current_price`.



This late initialization means that when liquidity is added, the `lower_tick` of the new liquidity position is inadvertently set to the `current_tick` at the moment `add_liquidity` is invoked, rather than its intended value derived from `current_price` initial state.

Impact

This fundamental misconfiguration leads to a mismatch between `current_tick` and the actual `current_price`. Since `current_tick` is not correctly aligned with the true `current_price`, subsequent swap operations will rely on an erroneous tick reference. This can result in swap calculations being inaccurate, leading to incorrect input/output amounts, failure to execute swaps, or swaps executing outside of expected price ranges, ultimately compromising the integrity and reliability of all trading activities within the system.

Recommendation

Refactor the initialization logic for `current_tick`. Ensure that `current_tick` is properly set during the initialization of `current_price`, before any liquidity addition functions are called. Verify that `upsert_boundary_tick` do not override or re-initialize `current_tick` in a manner that deviates from its intended initial state when handling liquidity boundaries.

Mitigation Review Log

Fixed in commit 370b8f56307e3728d8ac08bc817ab21d3b7a3d50.

4.14 Incorrect `fee_growth_outside` Initialization

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

During the initialization of a new tick within the `upsert_boundary_tick` function, the `fee_growth_outside_pt` and `fee_growth_outside_sy` values are incorrectly set. Specifically, they are both initialized using `self.fee_growth_index_global_pt` and `self.fee_growth_index_global_sy` respectively, regardless of the tick's position relative to `tick_current`.

According to the Uniswap V3 design principles, `fee_growth_outside` values should only be initialized with the global fee growth index if the new `tick` is less than or equal to `tick_current`. If `tick` is greater than `tick_current`, `fee_growth_outside` should be initialized to zero. The current implementation deviates from this design, potentially leading to miscalculations of fee growth for positions.



Impact

This bug can lead to inaccurate calculations of accrued fees for liquidity providers, particularly for positions that span across a newly initialized tick. If a tick is initialized with incorrect `fee_growth_outside` values, the `fee_growth_inside` calculation for positions that cross this tick will be skewed, resulting in an incorrect distribution of fees. This could lead to:

- Under-accrual or over-accrual of fees for LPs: Affecting their profitability and the fairness of the fee distribution mechanism.
- Inconsistent state: The protocol's internal state regarding fee accounting may become inconsistent with the intended Uniswap V3 logic.

Recommendation

Modify the `upsert_boundary_tick` function to correctly initialize `fee_growth_outside_pt` and `fee_growth_outside_sy` based on the position of `tick` relative to `tick_current`.

Mitigation Review Log

Fixed in commit `ef115db8e1d372e7f617631a4ee4940c72196593`.

4.15 Incorrect Subtree Sum Update During Tree Rotations

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

The `tick.recompute_upwards` function is designed to update subtree sums, representing the sum of liquidity deltas, by traversing upwards from `start_idx`. However, the current implementation fails to account for potential rotations within the underlying red-black tree. When a tree rotation occurs, a fundamental operation to maintain balance, the parent-child relationships and thus the composition of subtrees change. If the `recompute_upwards` logic does not specifically handle these structural changes, the recalculated subtree sums will not accurately reflect the new arrangement of nodes and their respective liquidity deltas.

Impact

This oversight directly leads to erroneous calculations of liquidity within specific price ranges or intervals. When subtree sums are incorrectly updated post-rotation, any subsequent query for aggregated liquidity across a range will yield a flawed result. This



impacts the accuracy of swap calculations, liquidity provision assessments, and overall price discovery, as the protocol's internal representation of available liquidity across ticks becomes inconsistent with the actual state.

Recommendation

The `tick.recompute_upwards` function must be updated to correctly handle subtree sum recomputation in the event of tree rotations. This may involve:

1. Integrating the subtree sum update directly into the rotation operation itself, ensuring the sums are adjusted as parent-child pointers change.
2. Ensuring that `recompute_upwards` is aware of the tree's structural changes and can correctly identify the new parent-child relationships and subtree boundaries after a rotation to accurately aggregate liquidity deltas.

This fix is crucial for maintaining the integrity of liquidity calculations within the tree structure.

Mitigation Review Log

Fixed in commit `de9ffe0c1579d144dbd9c914e5b16aaa68fc6eb8`.

4.16 Inconsistent Emissions Tracking Mechanism in `deposit_liquidity IX`

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

Within the `deposit_liquidity IX` in PR 2864, the process for updating emission trackers exhibits an inconsistency. Specifically, `index` and `last_seen_staged` in `market.emissions.trackers` are updated using `market.sy_balance`. However, the corresponding user-specific `last_seen_index` and `staged` within `staged_all` are updated using `position.lp_balance`. This divergence in reference values for related emission tracking parameters creates a potential for miscalculation.

```
149     let sy_balance = ctx.accounts.market.financials.sy_balance;
150
151     ctx.accounts
152         .market
153         .update_emissions_from_position_state(&position_state,
154             sy_balance);
```



[solana/programs/exponent_clmm/src/instructions/market_three/deposit_liquidity.rs#L149-L153](#)

```
76     self.emissions
77         .ensure_trackers_and_earn_all(emission_indexes, self.lp_balance as
    → u64);
```

[solana/programs/exponent_clmm/src/state/position.rs#L76-L77](#)

A critical underlying issue is that the `market.financials.sy_balance` variable, which is used for market-level emission tracking, lacks proper updates across the protocol. It is only updated during `deposit_liquidity` but remains unchanged during `liquidity withdraw_liquidity` and `swap`. This means that `sy_balance` becomes stale very quickly, failing to reflect the true state of the market's liquidity.

Impact

This inconsistency in the emission tracking mechanism can lead to an inaccurate distribution of emissions. Users might receive an incorrect amount of rewards, either over-receiving or under-receiving, depending on the specific state and sequence of operations. This directly affects the fairness and integrity of the emissions distribution system, potentially causing user dissatisfaction and financial discrepancies.

Recommendation

Harmonize the update mechanism for emissions tracking within the `deposit_liquidity` IX. It is recommended to consistently use a single, relevant balance metric (either `sy_balance` or `lp_balance`) for updating both the market-level and user-level emission tracking parameters. If both metrics are necessary for different aspects, their relationship and application should be clearly defined and consistently applied to avoid discrepancies.

Ensure `market.financials.sy_balance` is consistently updated: Implement mechanisms to update `sy_balance` in all functions that modify the market's liquidity, including `swap` and `withdraw`, to ensure it always reflects the true and current liquidity.

Mitigation Review Log

Fixed in commit `fe608a04c6ca5b7c617883f251ca2c208c694f8d`.

4.17 New Ticks Start With Zero Principals and Do Not Split Existing Interval Principals

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic Error



Description

When adding liquidity creates new boundary ticks, the code:

- Initializes the new tick's `principal_pt` / `principal_sy` to 0.
- Treats the new boundary as if it didn't split an existing interval.

```
160 // Construct a fresh boundary tick
161 let tick = Tick {
162     liquidity_net: 0,
163     liquidity_gross: 0,
164     fee_growth_outside_pt: self.fee_growth_index_global_pt, //
    snapshot at creation
165     ln_implied_rate: (1.0 + (key as f64) / TICK_KEY_BASE_POINTS).ln(),
166     subtree_sum: 0, // will be recomputed upwards
167     principal_pt: 0,
168     principal_sy: 0,
169     fee_growth_outside_sy: self.fee_growth_index_global_sy,
170 };
```

[solana/programs/exponent_clmm/src/state/ticks.rs#L160-L170](#)

However, inserting a new boundary splits an existing half-open interval `[u_L, u_R)` into `[u_L, u_new)` and `[u_new, u_R)`. The principals that were previously tracked on the left boundary (`principal_pt`, `principal_sy`) of `[u_L, u_R)` must be redistributed between the two new intervals (and, for SY, by the effective-price integral used elsewhere). Failing to do so makes the new right-hand subinterval start with zero principals, breaking conservation and causing mis-attribution of future swap flows and removals.

Impact

- Accounting Drift: Future swaps and removals inside the newly created subinterval will read `principal_pt/sy = 0`, underpaying/overpaying traders and LPs.
- Range-Dependent Arbitrage: Attackers can create ticks to reset principals in favorable regions, then trade to extract value from mis-accounted ledgers.

Recommendation

We should split the `principal_pt/sy` from the old interval.

Mitigation Review Log

Fixed in commit 99b29695e74cbd89abc9d3af5c8d2f3e4370413b.



4.18 Fixed-size Tick Storage Enables DoS via Junk Tick Initialization

Severity: Low

Status: Acknowledged

Target: Smart Contract

Category: DoS Risk

Description

The on-chain tick storage is fixed-size. An attacker can initialize (or cause initialization of) many ticks at “nonsensical” or far-away indices until the structure is full. Once saturated, honest LPs cannot create ticks within economically relevant ranges, effectively preventing new liquidity from being added where it’s needed.

Because tick creation appears cheap, the attacker can grief the pool by consuming the limited tick slots with junk entries.

Impact

- Users cannot open new positions in normal price ranges once the tick array fills.
- Markets become illiquid near the active price, worsening slippage and breaking core functionality.

Recommendation

It is recommended to use rbtree that is not fix-size.

Mitigation Review Log

We defend against this by validating a minimum LP amount and by enforcing a minimum distance between the two nearest ticks. We also plan to add tick removal logic.

4.19 Incorrect Strip Amount Calculation in WrapperProvideLiquidity IX

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

In the `WrapperProvideLiquidity` IX function, the calculation of `calc_strip_amount` is fundamentally incorrect for a CLMM model. The current implementation mistakenly uses the token balance recorded within the `financials` struct to determine the amount of strips.

In a CLMM model, liquidity is not uniformly distributed across all prices. Instead, it is concentrated within specific ticks. Therefore, simply relying on the total token balance in `financials` for strip calculation does not accurately reflect the underlying liquidity required,



which are tied to specific price ranges or the overall market depth within those ranges. A correct calculation must take into account the tick distribution and the specific liquidity depth at relevant ticks, rather than just the total token balance.

Impact

This miscalculation directly prevents the full and accurate addition of liquidity by users. When `calc_strip_amount` returns an incorrect value, the subsequent operations to provide liquidity based on this figure will be constrained, leading to an inability to fully utilize the intended amount of capital within the specified concentrated liquidity range. This results in inefficient capital deployment and users being unable to establish their desired liquidity positions completely.

Recommendation

The `calc_strip_amount` function must be refactored to align with CLMM principles. It should utilize CLMM-specific formulas that accurately account for the liquidity value and the square root prices of the relevant ticks, reflecting the actual liquidity depth within the intended range. Direct reliance on `financials.token_balance` must be replaced with a calculation that considers tick-level liquidity.

Mitigation Review Log

Fixed in commit 5789a8979dbe66d8f6c3d3edcb67b4a9e9fb856e.

4.20 Incorrect `fee_growth_index_global` Update when Ticks Add Fee Growth

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

Within the `Tick.add_fee_growth_checked` function, there is an error in how global fee growth indices are updated based on trade direction. The current implementation incorrectly updates `fee_growth_index_global_pt` for both `SyToPt` and `PtToSy` trade directions. According to the expected logic, when a trade occurs in the `PtToSy` direction, the `fee_growth_index_global_sy` value should be updated, not `fee_growth_index_global_pt`.



```
415     match swap_direction {
416         SwapDirection::SyToPt => {
417             self.fee_growth_index_global_pt =
418                 self.fee_growth_index_global_pt.checked_add(delta).
419                     unwrap();
420         }
421         SwapDirection::PtToSy => {
422             self.fee_growth_index_global_pt =
423                 self.fee_growth_index_global_pt.checked_add(delta).
424                     unwrap();
425         }
426     }
```

[solana/programs/exponent_clmm/src/state/ticks.rs#L415-L424](#)

Impact

This bug leads to an inaccurate accumulation of global fee growth for SY tokens. While `fee_growth_index_global_pt` will be over-inflated due to double-counting from PtToSy trades, `fee_growth_index_global_sy` will remain stagnant or under-updated. This discrepancy directly impacts the downstream calculation of `fee_growth_outside` for ticks and consequently, the `fee_growth_inside` for liquidity positions. The result is an incorrect attribution of accrued SY fees to liquidity providers, leading to a misrepresentation of their earnings and potential imbalances in fee distribution.

Recommendation

The `Tick.add_fee_growth_checked` function must be corrected to conditionally update the appropriate global fee growth index. For SyToPt trades, `fee_growth_index_global_pt` should be updated. For PtToSy trades, `fee_growth_index_global_sy` must be updated instead. The logic needs to differentiate between the two directions and target the corresponding fee growth index.

Mitigation Review Log

Fixed in commit 64f910488ab3be1da333bb5776063c85e0c7733b.

4.21 Missing mint_yt Validation During Market Initialization

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Data Validation



Description

During the market initialization process, there is a critical omission: the `mint_yt` parameter is not being validated. This lack of validation allows for the configuration of an incorrect `mint_yt` address. Without proper checks, the market can be brought online with a faulty `mint_yt` setting.

Impact

An improperly set `mint_yt` address directly renders all YT related trading functionalities inoperable. Since `mint_yt` is essential for the management of YT, any attempt to trade YTs will fail or revert if the address is invalid or points to a non-functional entity. This effectively cripples a core component of the market, preventing users from engaging with YT positions and undermining the intended protocol mechanics involving YT.

Recommendation

Implement robust validation for the `mint_yt` parameter during market initialization. Specifically, this validation should ensure that the `mint_yt` address is correctly configured and matches the expected `mint_yt` setting within the associated vault. This check is crucial to guarantee the correct interaction between the market and its underlying vault, thereby ensuring the full functionality of YT-related operations.

Mitigation Review Log

Fixed in commit 6106f7b6154d78830f2e9ab9e98d38de37a60328.

4.22 Unused PT and SY Token Transfers During Initialization

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

The current initialization process involves transferring PT and SY into the escrow. However, in a CLMM model, initial liquidity provision at the time of pool creation is typically not required in the same way it might be for other AMM designs. The transferred tokens are not subsequently used to establish corresponding liquidity positions. This means that capital is being moved into the system but remains idle, failing to contribute to the pool's liquidity.



Impact

This practice leads to inefficient capital allocation. Funds that are transferred but not utilized for liquidity provision are essentially locked without serving their intended purpose. This can negatively impact the overall capital efficiency of the protocol, as these tokens could otherwise be generating yield or participating in other productive activities. It also creates an unnecessary step in the initialization process that complicates resource management.

Recommendation

It is recommended to remove the unnecessary transfer of PT and SY tokens during the initialization phase if these tokens are not immediately used to establish active liquidity positions within the CLMM. If the intention is for these tokens to eventually become liquidity, the initialization process should be updated to either defer these transfers until they are actively deployed as liquidity or integrate a clear mechanism for their immediate utilization upon transfer.

Mitigation Review Log

Fixed in commit e6c60801ecbce1f772685c2056822ccab1370f4f.

4.23 Informational and Undetermined Issues

RemoveMarketEmission Lacks Validation

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Data Validation

The `RemoveMarketEmission` function allows an administrator to remove a market emission. If an administrator proceeds with removal while active `lp_positions` are still configured with that `emission_index`, these `lp_positions` will become corrupted, as their `emission_index` will point to an invalid or non-existent entry.

Emission Processing Lacks Specific Support for Token2022 Transfer Fees and Memo

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Code QA

The current emission processing mechanism does not explicitly account for advanced features present in Token2022 standards, such as `transfer_fee` and `memo`. While not inherently breaking, the system's logic appears to treat all tokens as standard Token without making provisions for these additional considerations that Token2022 tokens might enforce.



Unhandled `is_current_flash_swap` Flag in Swap Logic

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Code QA

The `SwapArgs` structure includes a boolean field named `is_current_flash_swap`. This field suggests an intention to differentiate and potentially apply specific logic for flash swap operations. However, upon reviewing the codebase, this `is_current_flash_swap` field is not actively referenced or used within the swap functions. This means that flash swaps are processed under the same logic path as regular swaps, without any specialized handling that might be necessary for their unique characteristics.

Incorrect Calculation of Additional Rent for Account Resizing

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Logic Error

The following line of code is responsible for calculating the additional rent required when resizing an account:

```
82     let additional_rent = Rent::get()?.minimum_balance(new_size -  
        old_size);
```

[solana/programs/exponent_clmm/src/instructions/market_three/admin/modify_market_setting.rs#L82-L82](#)

This calculation attempts to determine the rent for the difference in size (`new_size - old_size`). However, the `minimum_balance` function on the `Rent` struct calculates the rent required for the total size provided as its argument to keep an account rent-exempt for that specific size. It does not calculate the incremental rent needed for a size increase.

The correct way to calculate the additional rent for a size increase is to find the minimum balance for the `new_size` and subtract the minimum balance for the `old_size`.

Division by Zero in Percentage Increase Calculation

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: DoS Risk

The calculation of `percentage_increase` involves division by `start_balance`. A critical vulnerability exists when `start_balance` is zero. This situation is particularly likely immediately after protocol initialization, or if `window_duration_seconds` (which is initially 0 and administrator-configurable) prevents `start_balance` from accumulating a non-zero value. If `start_balance` is 0 (and `balance_increase > 0`), the division `balance_increase / 0.0` results in `f64::INFINITY`. Casting `f64::INFINITY` to `u32` deterministically yields `u32::MAX`. This value will inevitably exceed `self.max_net_balance_change_positive_percentage`, causing the `require!` check to always fail and revert the transaction.



lp_position Initialization Keyword Restricts Multiple Liquidity Deposits

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Logic Error

In the current `deposit_liquidity` implementation, the `lp_position` account is marked with the `init` keyword. This implies that the `lp_position` can only be initialized once. Consequently, it is not possible to add additional liquidity to an already existing `lp_position` using this function. While this might be an intentional design choice to enforce immutable liquidity positions, it significantly restricts user flexibility. If the intention is to allow users to incrementally add liquidity to an existing position, the `init` keyword is inappropriate for `lp_position` and prevents such functionality. This design decision impacts capital efficiency and user experience for liquidity providers who may wish to manage their positions over time without creating new ones for each deposit.

User lp_position Accounts Are Never Closed

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Logic Error

The current protocol design for user `lp_position` accounts does not include a function or mechanism to close these accounts. Once an `lp_position` account is created, it persists on-chain indefinitely, even after all associated liquidity has been withdrawn. Without a closing mechanism, the SOL paid for rent for these `lp_position` accounts is effectively locked away from the user permanently, even when the account is no longer serving its purpose.

Incorrect SIZEOF for ConfigurationOptions

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Logic Error

`ConfigurationOptions::SIZE_OF` hard-codes wrong field sizes:

Correct serialized length: $8 + 2 + 8 + 8 + 8 + 4 + 32 + 32 = 102$ bytes

Current code uses: $8 + 2 + 8 + 8 + 16 + 2 + 64 = 108$ bytes

This overestimates by +6 bytes.

Over-allocation on `realloc : MarketThree::size_of` will allocate more space than required for the account. That wastes rent and can push the account toward Solana's account size limits earlier than necessary (potentially causing otherwise valid resizes to fail).



5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

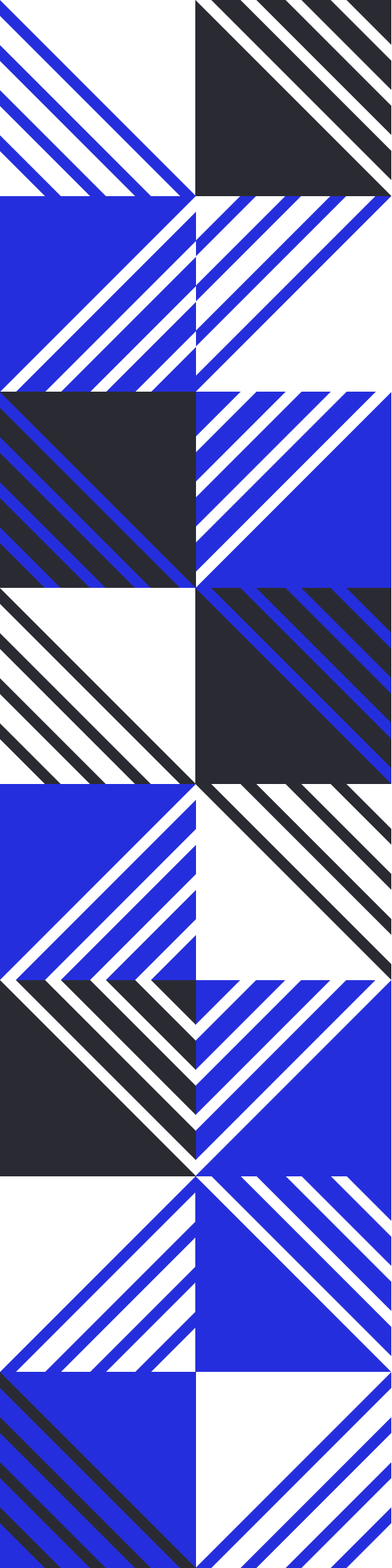
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



 <https://offside.io/>

 <https://github.com/offsidelabs>

 https://twitter.com/offside_labs